Smart Asset Management Platform (SAMP)

Technical Project Documentation

IIT Roorkee Cultural Council

June 12, 2026

Contents

1 Problem Understanding

Managing high-value cultural assets—such as media equipment, professional lighting grids, stage props, and audio infrastructure—across multiple campus clubs and student sections presents a significant logistical challenge. The traditional methods of tracking inventory rely on manual ledgers or fragmented spreadsheets, leading to severe inefficiencies.

Key Challenges Addressed:

- **Inventory Fragmentation:** Lack of a centralized, real-time view of what equipment is available, currently in use, or overdue for return.
- **Scheduling Conflicts:** Without a visual lookahead calendar, double-booking highly demanded gear is common, leading to operational bottlenecks during campus events.
- **Accountability & Security:** Unregulated physical handovers make it difficult to track who has what item. There is a critical need for verified checkout and return processes.
- **Authorization Overheads:** Managing access tiers between standard students, club administrators, and overseeing faculty members requires a streamlined, digital approval pipeline.

The Smart Asset Management Platform (SAMP) solves these issues by providing a unified web workspace. It automates inventory balancing, visually prevents scheduling conflicts, enforces Role-Based Access Control (RBAC), and secures physical equipment handovers using cryptographic QR tokens.

2 System Architecture

SAMP utilizes a streamlined, decoupled Monolithic Architecture. It pairs a lightweight, RESTful Node.js backend with a highly reactive, vanilla HTML5 frontend, intentionally avoiding heavy frontend frameworks or external database dependencies to ensure rapid local deployment and high portability.

- **Frontend (Client Tier):** Built with Vanilla JavaScript (ES6), HTML5, and CSS3. It leverages native Web APIs (like WebRTC for webcam access) and lightweight libraries (Chart.js for analytics, Html5-QRCode for scanning).
- **Backend (Application Tier):** An Express.js server handles routing, cryptographic token validation (JWT), role-based middleware interception, and business logic execution.
- **Data Engine (Persistence Tier):** A custom, synchronous In-Memory Object Proxy engine intercepts JavaScript array mutations and automatically flushes state changes to a flat JSON file (`db_store.json`), guaranteeing atomic persistence without an external database daemon.

3 Database Schema

The application relies on a localized NoSQL-style JSON document store containing three primary arrays: `users`, `assets`, and `bookings`.

3.1 Users Collection

Stores credential and authorization data.

- `id` (String): Unique identifier (e.g., 'u_admin').
- `name` (String): Full name or Club Section Name.
- `email` (String): Campus email used for authentication.
- `password` (String): Plaintext or bcrypt-hashed secret.
- `role` (String): ENUM [`consumer`, `admin`, `faculty`, `pending_admin`].

3.2 Assets Collection

The global registry of borrowable equipment.

- `id` (String): Unique asset identifier (e.g., 'a_1630...').
- `name` (String): Human-readable item name.
- `category` (String): ENUM [`Media`, `Lighting`, `Audio`, `Props`].
- `description` (String): Brief overview of the asset.
- `totalQuantity` (Integer): Total physical units owned by the repository.
- `availableQuantity` (Integer): Real-time calculated pool available for checkout.
- `imageUrl` (String): Base64 encoded string from webcam capture.
- `isArchived` (Boolean): Soft-delete flag.

3.3 Bookings Collection

The transactional ledger tracking the borrowing lifecycle.

- `id` (String): Transaction identifier.
- `userId` (String): Reference to the requesting user.
- `assetId` (String): Reference to the target equipment.
- `quantity` (Integer): Number of units requested.
- `startDate` / `endDate` (Date Strings): The reserved calendar window.
- `status` (String): ENUM [`Pending`, `Approved`, `Issued`, `Returned`, `Rejected`].
- `adminMessage` (String): Optional context attached to rejections.

4 Entity Relationship Diagram (ERD)

Below is the logical relationship mapping between the core entities in the platform.

A single **User** can generate many **Bookings** over time. Similarly, a single **Asset** can be tied to many distinct **Bookings** (distinguished by temporal boundaries).

5 API Overview

The backend exposes a RESTful API protected by Bearer JWT authorization.

5.1 Authentication & User Management

- `POST /api/auth/register`: Creates a new user profile or initiates an admin candidate access sequence.
- `POST /api/auth/login`: Validates credentials and provisions a JWT session token.
- `GET /api/users/pending-admins`: *[Faculty]* Fetches users requesting administrative privileges.
- `PATCH /api/users/:id/resolve-role`: *[Faculty]* Approves or rejects admin applications.

5.2 Asset Ledger Commands

- `GET /api/assets`: Returns the dynamic catalog, calculating available stock for the current day.
- `GET /api/assets/:id/availability`: Compiles a 60-day visual status timeline framework to populate the frontend calendar engine.
- `POST /api/assets`: *[Admin]* Ingests a new asset record, saving Base64 webcam data.
- `PUT /api/assets/:id`: *[Admin]* Edits core asset parameters (prevents decreasing stock below currently checked-out thresholds).
- `DELETE /api/assets/:id`: *[Admin]* Toggles the `isArchived` flag for soft deletion.

5.3 Booking Lifecycle Controls

- `GET /api/bookings/check-availability`: Sweeps backend arrays to isolate allocation bottlenecks across a specific requested date range.
- `POST /api/bookings/request`: Enqueues a reservation frame.
- `GET /api/bookings/my-bookings`: Pulls the transactional history for the authenticated consumer.
- `PATCH /api/bookings/:id/approval`: *[Admin]* Transitions status to `Approved` or `Rejected`.
- `PATCH /api/bookings/:id/issue`: *[Admin]* Flags item as physically handed over.
- `PATCH /api/bookings/:id/return`: *[Admin]* Closes the active contract and replenishes available stock.

6 Design Decisions

Several critical architectural choices were made to optimize the platform for the specific needs of a university cultural council environment.

1. **In-Memory Proxy Database Engine:**
   Instead of configuring a heavy external database (like PostgreSQL or MongoDB), the platform uses JavaScript ES6 `Proxy` objects. This intercepts array mutations (`push`, `splice`, etc.) and triggers an immediate, synchronous write to `db_store.json`. This provides the ease of manipulating raw arrays in memory while guaranteeing instant crash-proof persistence, ideal for a micro-deployment.

2. **Algorithmic Overlap Prevention:**
   Rather than simply tracking current stock, the availability engine uses a sliding 60-day window loop. It calculates maximum depletion on *every individual day* within a requested range. This completely eliminates the possibility of double-booking assets that are returned and re-issued within the same week.

3. **Base64 Hardware Native Logging:**
   The platform natively hooks into the user's webcam via WebRTC to snap photos of new inventory. To avoid the complexity and cost of external object storage buckets (like AWS S3), the application encodes the HTML5 Canvas output as a compressed Base64 Data URL and saves it directly within the JSON text ledger.

4. **QR Code Handover Protocols:**
   To bridge the digital system with physical reality, approved bookings generate standard QR matrices containing the transaction ID. Administrators scan this at the physical repository counter via the `Html5-QRCode` library, allowing for rapid 1-click *Issue* workflows without manual typing.

5. **Vanilla Client Independence:**
   The frontend completely avoids frameworks like React or Angular. By relying on Vanilla JS, native DOM manipulation, and CSS variables, the application requires zero build steps (no Webpack/Babel). This ensures legacy compatibility, incredibly fast page loads, and makes it trivial for future student developers to maintain or host on minimal-tier university servers.